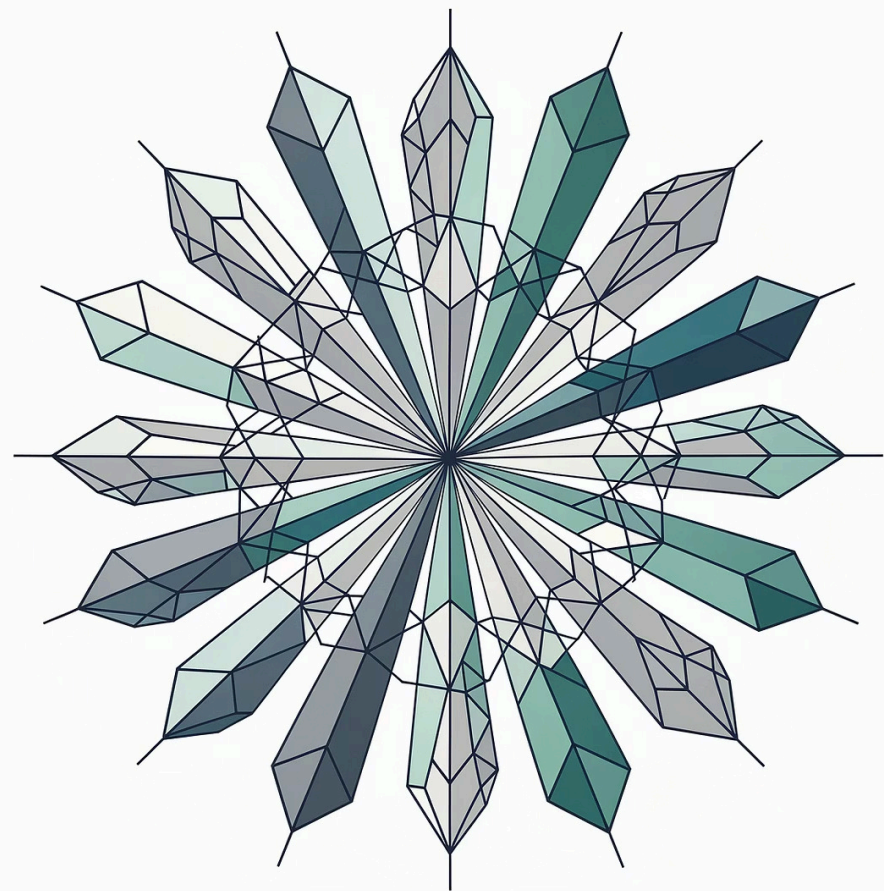




That netitiongert - necioge your lilistien
fof piongy your opaflienig the.
everyg thone shiorte theyenling sniart.

MST Algorithms — Advanced Module

A deep dive into **Prim's Algorithm** and **Kruskal's Algorithm** — two foundational greedy approaches to constructing Minimum Spanning Trees, with full complexity analysis, step-by-step traces, and real-world applications.

GRAPH ALGORITHMS

ADVANCED DSA

GREEDY ALGORITHMS



PART 1: Prim's Algorithm

Prim's algorithm grows the Minimum Spanning Tree one edge at a time, starting from a single vertex and always expanding via the cheapest available edge.

Section 1: Topic Classification

Topic Details

Topic Name: Prim's Algorithm

Category: Graph Algorithms / Greedy Algorithms

Domain: Computer Science / Minimum Spanning Tree

Difficulty: Advanced DSA

Prerequisites

- Weighted Graphs
- MST Theory (Cut Property)
- Priority Queues (Min-Heap)
- Adjacency List representation

Industry Relevance

Network infrastructure design, VLSI layout, cluster analysis, real-time MST construction on dynamic graphs.

Recommended Path

MST Theory → [Prim's Algorithm](#) → Kruskal's Algorithm

Section 2: The Core Idea

"The MST grows like a crystal. Start with one atom (vertex). At each step, attach the nearest atom not yet in the crystal."

Intuition: Grow One Tree from a Seed

Prim's algorithm builds the MST by starting with a single vertex and **greedily expanding** the tree one edge at a time. At each step, it adds the **cheapest edge** connecting the current tree to a vertex not yet in the tree.

How It Applies the Cut Property

At every step, the growing tree T defines a cut (S = vertices in T , $V \setminus S$ = vertices not in T). Prim's always picks the minimum weight edge crossing this cut. By the **Cut Property**, that edge is always safe to add to the MST.

1

Invariant

Edges selected so far always form a tree

2

Growth

The tree grows by exactly one edge each iteration

3

Safety

Each added edge is the minimum crossing the current cut (S , $V \setminus S$)

4

Result

By the Cut Property, each edge is safe $\rightarrow$ final result is MST

Section 3: The Algorithm

Conceptual Pseudocode

```
PRIM(G, w, r): // r = starting vertex
  FOR each vertex  $u \in V$ :
     $key[u] = \infty$  // cheapest edge to tree
     $parent[u] = \text{NULL}$ 
     $in\_mst[u] = \text{FALSE}$ 
   $key[r] = 0$ 
   $Q = \text{min-priority queue (all vertices, keyed by key[])}$ 

  WHILE  $Q$  is not empty:
     $u = \text{EXTRACT-MIN}(Q)$ 
     $in\_mst[u] = \text{TRUE}$ 
    FOR each  $(v, w)$  in  $adj[u]$ :
      IF  $v \in Q$  AND  $w < key[v]$ :
         $parent[v] = u$ 
         $key[v] = w$ 
         $\text{DECREASE-KEY}(Q, v, w)$ 

  RETURN  $\{(parent[u], u) : u \in V, u \neq r\}$ 
```

Binary Min-Heap Implementation (Lazy Deletion)

```
FUNCTION prim(graph, V, start=0):
   $key = [INF] * V$ 
   $parent = [-1] * V$ 
   $in\_mst = [FALSE] * V$ 
   $key[start] = 0$ 
   $heap = [(0, start)]$ 

  WHILE heap not empty:
     $(k, u) = \text{heappop}(heap)$ 
    IF  $in\_mst[u]$ : CONTINUE // stale entry
     $in\_mst[u] = \text{TRUE}$ 
    FOR  $(v, w)$  in  $graph.adj[u]$ :
      IF NOT  $in\_mst[v]$  AND  $w < key[v]$ :
         $key[v] = w$ 
         $parent[v] = u$ 
         $\text{heappush}(heap, (w, v))$ 

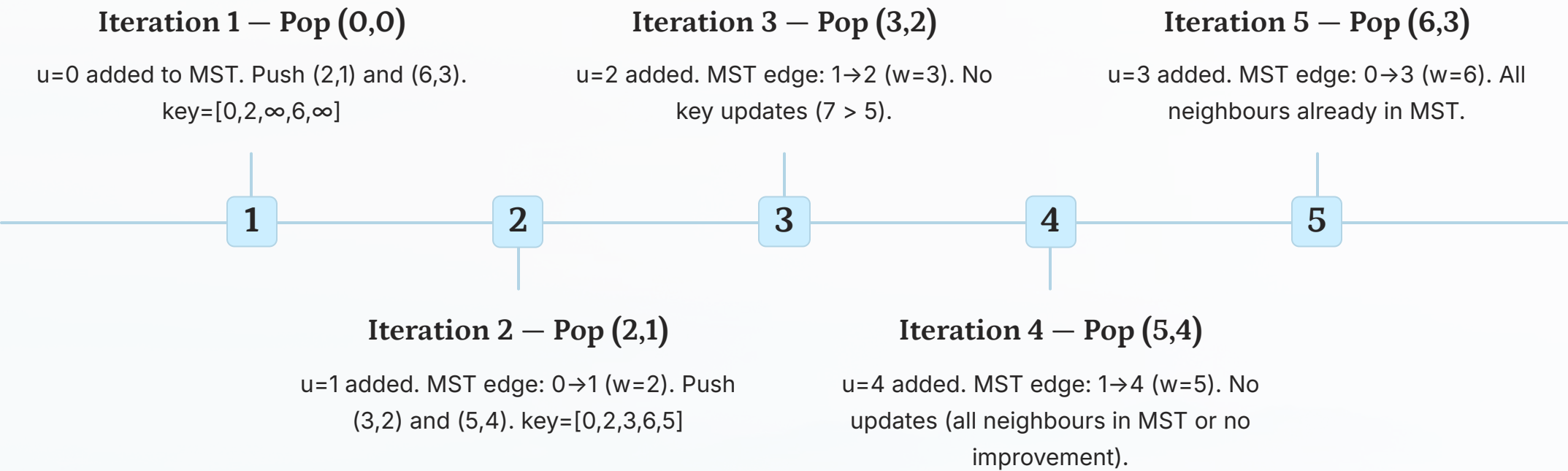
   $mst\_edges = [(parent[v], v, key[v])$ 
    for  $v$  in  $range(V)$  if  $parent[v] \neq -1]$ 
   $mst\_weight = \text{sum}(key[v] \text{ for } v \text{ in } range(V)$ 
    if  $v \neq start)$ 
  RETURN  $mst\_edges, mst\_weight$ 
```

Lazy Deletion: Stale heap entries are not removed. When a vertex is popped, check `in_mst[u]` and skip if already processed. This simplifies implementation while maintaining correctness.

Section 4: Step-by-Step Trace

Graph: Vertices {0,1,2,3,4} — Starting at vertex 0

Edges: (0,1,2), (0,3,6), (1,2,3), (1,3,8), (1,4,5), (2,4,7), (3,4,9)



Final MST Edges

MST = {(0,1,2), (1,2,3), (1,4,5), (0,3,6)}

Total MST weight = 2 + 3 + 5 + 6 = 16

MST structure:

0

/ \

2 6

/ \

1 3

/ \

3 5

| |

2 4

Key Array Evolution

After Iter.	key[0]	key[1]	key[2]	key[3]	key[4]
Init	0	∞	∞	∞	∞
1	0	2	∞	6	∞
2	0	2	3	6	5
3	0	2	3	6	5
4	0	2	3	6	5

Section 5: Complexity Analysis

Priority Queue Comparison

Operation	Binary Heap	Fibonacci Heap
EXTRACT-MIN	$O(\log V)$	$O(\log V)$ amortised
DECREASE-KEY	$O(\log V)$	$O(1)$ amortised
Total EXTRACT-MIN	$O(V \log V)$	$O(V \log V)$
Total DECREASE-KEY	$O(E \log V)$	$O(E)$
TOTAL	$O((V+E) \log V)$	$O(E + V \log V)$

When Each Variant Wins

Sparse ($E \approx V$)

Binary Heap: $O(V \log V)$ — both variants similar

Dense ($E \approx V^2$)

Fibonacci Heap: $O(V^2)$ wins over Binary Heap $O(V^2 \log V)$

Very Dense — Array Scan

$O(V^2)$ array-based Prim's beats all heap variants: $O(V^2)$ vs $O(V^2 \log V)$

Array-Based Prim's — $O(V^2)$ for Dense Graphs

```
FUNCTION prim_dense(matrix, V):
  key = [INF] * V; parent = [-1] * V; in_mst = [FALSE] * V
  key[0] = 0
  FOR _ in range(V):
    u = argmin(key[v] for v in range(V) if NOT in_mst[v]) // O(V) scan
    in_mst[u] = TRUE
    FOR v in range(V):
      IF matrix[u][v] != INF AND NOT in_mst[v]:
        IF matrix[u][v] < key[v]:
          key[v] = matrix[u][v]; parent[v] = u
  RETURN [(parent[v], v, key[v]) for v in range(1, V)]
// Time:  $O(V^2)$  Space:  $O(V)$  Better than  $O((V+E) \log V)$  when  $E = \Theta(V^2)$ 
```



Section 6: Prim's vs Kruskal's Preview

Prim's Algorithm

Grows **one connected component** from a starting vertex. At each step, picks cheapest edge leaving current component. Priority queue stores vertices keyed by cheapest edge to tree.

Better for:

- Dense graphs (adjacency matrix + $O(V^2)$ variant)
- Graphs where you want to start from a specific vertex
- Streaming / online graph scenarios

Kruskal's Algorithm

Processes **ALL edges globally** in weight order. Builds MST by merging components with Union-Find. At each step, picks globally cheapest edge that doesn't form a cycle.

Better for:

- Sparse graphs
- When edges are already sorted or nearly sorted
- Parallel / distributed MST computation

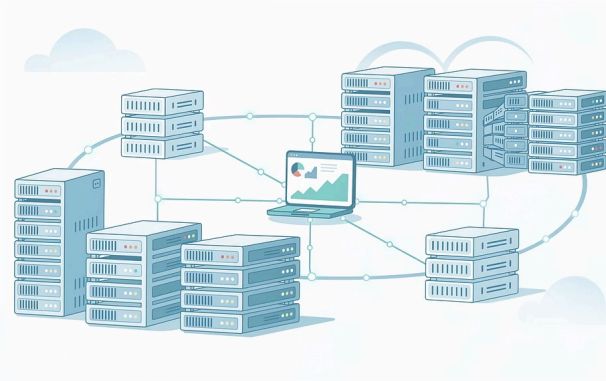
Section 7: Real-World Use Cases



Network Cable Laying (ISP Infrastructure)

An ISP must lay fibre optic cables to connect 20 data centres. Each potential cable route has an installation cost (weight). Find the network with minimum total cable cost.

Why Prim's: Data centres form a **dense graph** (almost every pair has a potential route). Prim's $O(V^2)$ variant on the adjacency matrix is faster than Kruskal's $O(E \log E)$ since $E \approx V^2$. Start from the highest-traffic hub for early expansion toward important nodes.



Real-Time MST Updates

A distributed monitoring system builds a spanning tree of server nodes. Nodes join and leave dynamically.

Why Prim's: Prim's naturally handles **incremental tree growth** — when a new vertex v arrives with edges to existing tree vertices, simply add v with its minimum-weight edge to the tree. No need to reprocess all edges as Kruskal's would require.

Section 8: Hands-On Labs

Lab 1: Implement Prim's with Both Priority Queue Variants

Graph: 6 vertices, edges: (0,1,4), (0,2,3), (1,2,1), (1,3,2), (2,3,4), (2,4,5), (3,4,6), (3,5,7), (4,5,8)

1. Implement `prim_heap(adj_list, V)` using a min-heap (lazy deletion)
2. Implement `prim_dense(matrix, V)` using $O(V^2)$ array scan
3. Verify both produce the same MST weight
4. Count: how many DECREASE-KEY operations does the heap version perform?
5. For this graph ($E=9$, $V=6$): which variant is theoretically faster? Calculate and verify.

✓ **Expected MST:** Verify edges include (1,2,1), (1,3,2), (0,2,3), (0,1,4) — confirm total weight.

Lab 2: Prim's on a Grid Graph

Grid: 4×4 grid where each cell is a vertex. Adjacent cells (horizontal/vertical) have random weights between 1 and 20.

1. Generate a random 4×4 weighted grid graph (16 vertices, up to 24 edges)
2. Run Prim's starting from vertex (0,0) (top-left)
3. Visualise the MST by marking selected edges on the grid
4. Vary the start vertex: does the MST change? Does the total weight change?
5. Find ALL MSTs if any edge weights are equal (hint: use lexicographic tie-breaking)

Section 16: Common Mistakes – Prim's

Marking in_mst Before Processing Neighbours

Impact: Some neighbours never get their key updated.

Fix: Mark `in_mst[u] = TRUE` AFTER extracting `u`, BEFORE processing `u`'s neighbours.

Not Skipping Stale Heap Entries

Impact: Same vertex processed multiple times.

Fix: Check IF `in_mst[u]`: CONTINUE immediately after `heappop`.

Updating `key[v]` Without Pushing to Heap

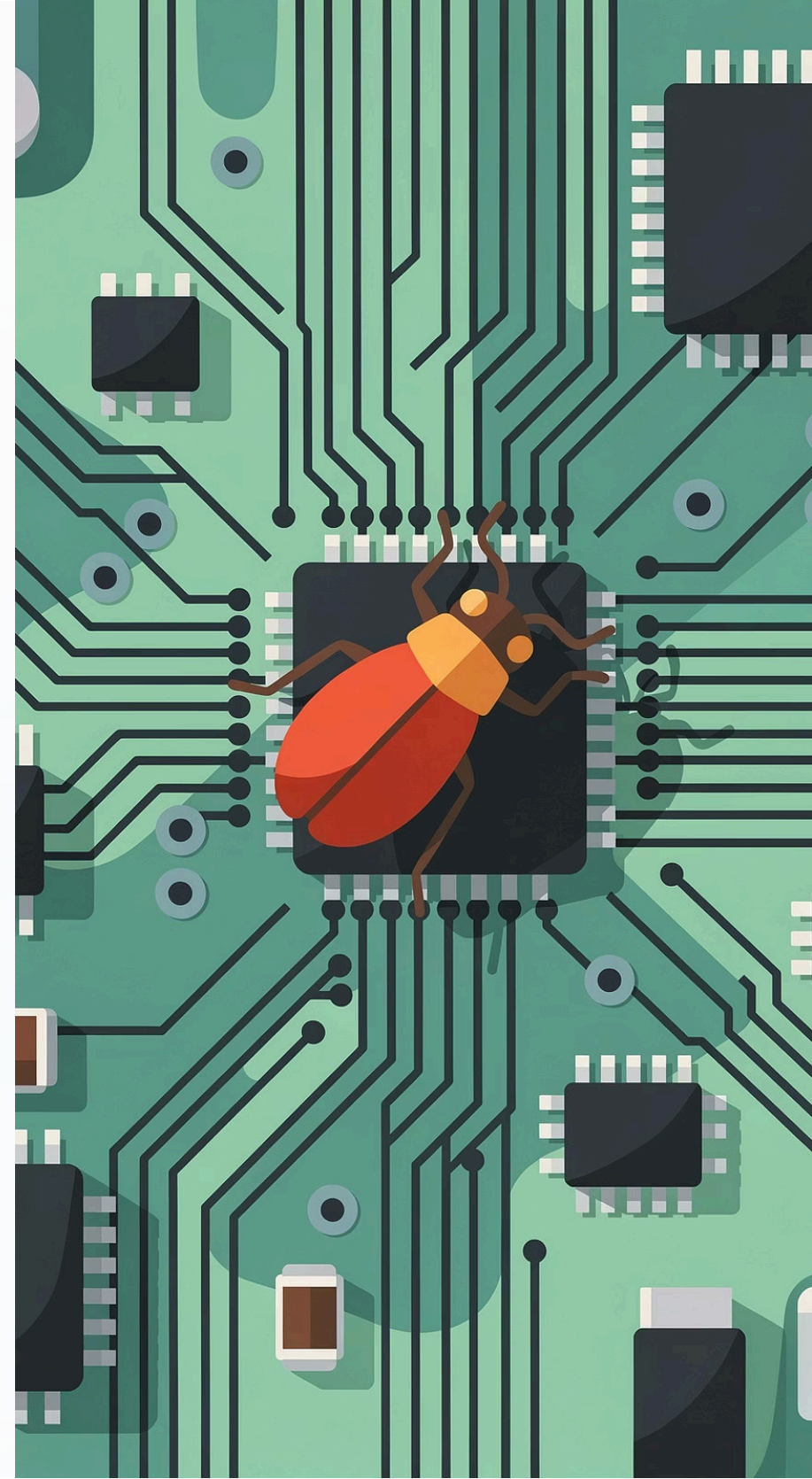
Impact: `key[v]` updated in array but heap not notified.

Fix: Always `heappush(heap, (new_key, v))` after updating `key[v]`.

Using Prim's on a Disconnected Graph

Impact: MST computed for one component only.

Fix: After Prim's, verify all `in_mst[v] == TRUE`. For forests: outer loop FOR `v` in `V`: IF NOT `in_mst[v]`: `prim_from(v)`.



Section 18: Prim's Learning Summary

1 Grow One Tree

Prim's builds the MST by growing one connected tree from a start vertex — extract minimum-key vertex, update neighbours' keys.

3 Correctness via Cut Property

Each extracted edge is the light edge across cut (in_mst, not_in_mst) → guaranteed safe by the Cut Property.

2 Key & Parent Arrays

$\text{key}[v]$ = weight of cheapest edge connecting v to the current tree. $\text{parent}[v]$ = which tree vertex offers that cheapest edge.

4 Complexity by Variant

Binary heap: $O((V+E) \log V)$ for sparse graphs. Array scan: $O(V^2)$ for dense graphs. Fibonacci heap: $O(E + V \log V)$ asymptotically best.

📌 **Quick Reference:** Strategy = Grow one tree, always pick cheapest edge out. Loop: $u = \text{EXTRACT-MIN} \rightarrow \text{in_mst}[u] = \text{TRUE} \rightarrow \text{FOR each } (v,w) \text{ in } \text{adj}[u]: \text{IF NOT in_mst}[v] \text{ AND } w < \text{key}[v]: \text{key}[v]=w; \text{parent}[v]=u; \text{DECREASE-KEY or push.}$

PART 2: Kruskal's Algorithm

Kruskal's algorithm takes a global view — sorting all edges by weight and greedily merging components, using Union-Find to detect cycles in near-constant time.



Section 1: Topic Classification

Topic Details

Topic Name: Kruskal's Algorithm

Category: Graph Algorithms / Greedy Algorithms

Domain: Computer Science / Minimum Spanning Tree

Difficulty: Advanced DSA

Prerequisites

- Weighted Graphs
- MST Theory
- Union-Find / Disjoint Set Union (DSU)
- Edge sorting

Industry Relevance

Network design, image segmentation (graph cuts), single-linkage clustering, circuit design.

Recommended Path

MST Theory → Prim's Algorithm → [Kruskal's Algorithm](#)

Section 2: The Core Idea

"Scan all roads by cost, cheapest first. If a road connects two disconnected towns → build it. If both towns are already connected → skip it (building would create a redundant loop)."

Intuition: Global Cheapest Safe Edge

While Prim's grows one tree locally from a start vertex, Kruskal's takes a **global view**. It sorts all edges by weight and processes them one by one, adding each edge to the MST if and only if it doesn't create a cycle.

Applying Both MST Properties

Cycle Property

When Kruskal's **skips** an edge (u,v) because u and v are already connected → (u,v) would form a cycle. It's the heaviest edge in that cycle → Cycle Property says skip it. ✓

Cut Property

When Kruskal's **adds** edge (u,v) → u and v are in different components. The edge is the minimum weight edge crossing the cut between their components. ✓

Section 3: The Algorithm

Kruskal's Pseudocode

```
KRUSKAL(G, w):  
  A =  $\emptyset$   
  FOR each vertex  $v \in V$ :  
    MAKE-SET(v)  
  Sort E by weight (non-decreasing) //  $O(E \log E)$   
  FOR each edge (u, v, w) in sorted order:  
    IF FIND(u)  $\neq$  FIND(v):  
      A = A  $\cup$  {(u, v, w)}  
      UNION(u, v)  
    IF |A| == V - 1: BREAK  
  RETURN A
```

Union-Find (DSU) Operations

```
MAKE-SET(x): parent[x]=x; rank[x]=0
```

```
FIND(x): // Path compression
```

```
  IF parent[x]  $\neq$  x:  
    parent[x] = FIND(parent[x])  
  RETURN parent[x]
```

```
UNION(x, y): // Union by rank
```

```
  rx = FIND(x); ry = FIND(y)  
  IF rx == ry: RETURN  
  IF rank[rx] < rank[ry]: SWAP(rx, ry)  
  parent[ry] = rx  
  IF rank[rx] == rank[ry]: rank[rx] += 1
```

The Inverse Ackermann Function — $\alpha(n)$

With both path compression and union by rank, the amortised time per FIND or UNION is $O(\alpha(n))$ — a function that grows so slowly it is effectively constant for all practical purposes.

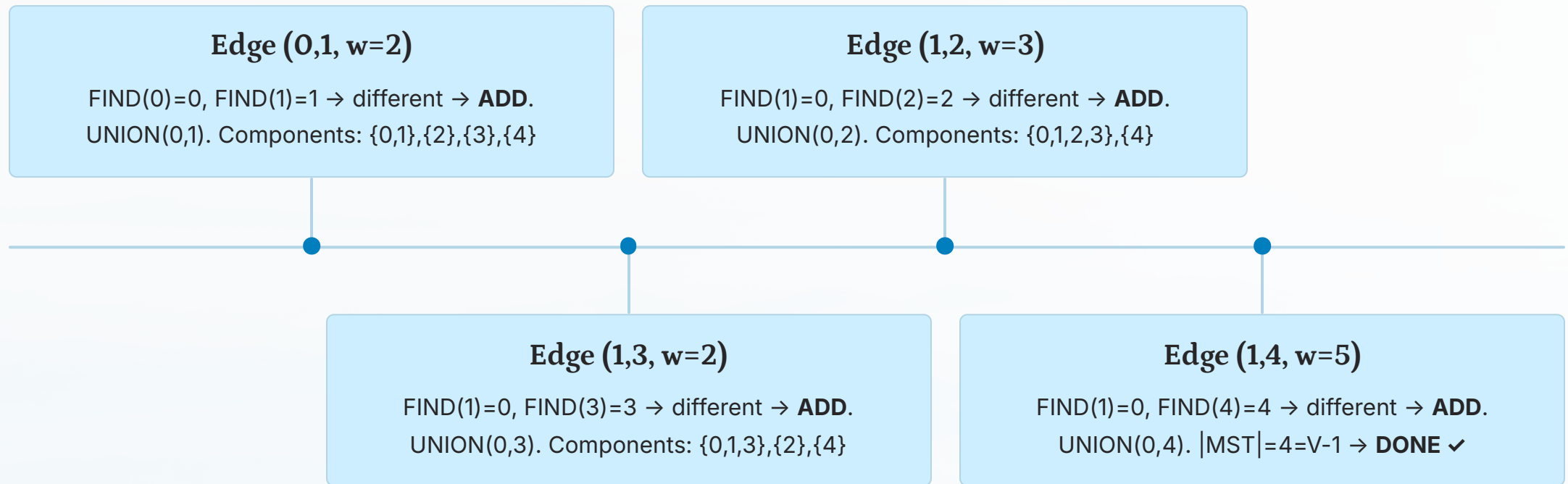
n	$\alpha(n)$
1	0
2	1
4	2
16	3
65,536	4
2^{65536}	5

 For any n that could ever exist in practice: $\alpha(n) \leq 4$. We treat DSU operations as $O(1)$ amortised.

Section 4: Step-by-Step Trace

Graph: Vertices {0,1,2,3,4} — Edges sorted by weight

Sorted edges: (0,1,2), (1,3,2), (1,2,3), (1,4,5), (0,3,6), (2,4,7), (1,3,8), (3,4,9)



Final Result

MST = {(0,1,2), (1,3,2), (1,2,3), (1,4,5)}

Total MST weight = 2 + 2 + 3 + 5 = 12

Remaining edges (0,3,6), (2,4,7), (1,3,8), (3,4,9)

→ all skipped (would form cycles)



Note on Prim's vs Kruskal's weights: Prim's trace gave weight 16 on a different graph. These are separate examples — both algorithms on the same graph will always give the same MST weight (though potentially different edge sets if weights are tied).

Section 5: Complexity Analysis

Kruskal's Complexity Breakdown

Step 1: Initialise Union-Find for V vertices: $O(V)$

Step 2: Sort E edges by weight: $O(E \log E)$

Note: $E \leq V(V-1)/2 \rightarrow \log E \leq 2 \log V$

So $O(E \log E) = O(E \log V)$

Step 3: Process E edges:

Each edge: 2 FIND + possibly 1 UNION

Each FIND/UNION: $O(\alpha(V)) \approx O(1)$ amortised

Total: $O(E \cdot \alpha(V)) \approx O(E)$

TOTAL: $O(E \log E) = O(E \log V)$

← dominated by sorting step

Space: $O(V)$ for Union-Find + $O(E)$ for edge list

Kruskal's vs Prim's

Scenario	Kruskal's	Prim's
Sparse ($E=O(V)$)	$O(V \log V)$	$O(V \log V)$
Dense ($E=O(V^2)$)	$O(V^2 \log V)$	$O(V^2)$ ← wins
Pre-sorted edges	$O(E \cdot \alpha(V)) \approx O(E)$ ← wins	$O((V+E) \log V)$



Pre-sorted edges change everything: If edges are already sorted (e.g., road network by distance, time-sorted event stream), Kruskal's runs in near-linear $O(E)$ vs Prim's $O(E \log V)$. A dramatic advantage in practice.

Section 6: Kruskal's for Minimum Spanning Forest

When the graph is **disconnected**, Kruskal's naturally produces a minimum spanning forest — one MST per connected component. No modification to the algorithm is needed.

```
FUNCTION kruskal_forest(graph, V):  
    // Same as standard Kruskal's — no changes needed!  
    // If graph has k components, result has V-k edges  
    parent = list(range(V))  
    rank = [0] * V  
    mst = []  
    FOR (u, v, w) in sorted(graph.edges, key=lambda e: e[2]):  
        ru = find(parent, u)  
        rv = find(parent, v)  
        IF ru != rv:  
            mst.append((u, v, w))  
            union(parent, rank, ru, rv)  
    components = count_distinct(find(parent, v) for v in range(V))  
    RETURN mst, components
```

Connected Graph

Result has **V-1 edges** — one spanning tree.

k-Component Graph

Result has **V-k edges** — a spanning forest of k trees.

- ❑ After Kruskal's, count distinct roots via FIND to determine the number of components. Prim's requires an explicit outer loop to handle disconnected graphs; Kruskal's handles it for free.

Section 7: Advanced Union-Find — DSU with Rollback

For problems requiring **undoing** Union operations (e.g., offline dynamic connectivity), use Union by Rank **without** path compression. Path compression makes rollback impossible; rank alone still gives $O(\log n)$ per operation.

```
FUNCTION union_with_rollback(parent, rank, x, y):
    rx = find_no_compression(parent, x)
    ry = find_no_compression(parent, y)
    IF rx == ry: RETURN None
    IF rank[rx] < rank[ry]: SWAP(rx, ry)
    // Save state for rollback
    rollback_stack.append((ry, parent[ry], rx, rank[rx]))
    parent[ry] = rx
    IF rank[rx] == rank[ry]: rank[rx] += 1
    RETURN (rx, ry)

FUNCTION rollback():
    (ry, old_parent_ry, rx, old_rank_rx) = rollback_stack.pop()
    parent[ry] = old_parent_ry
    rank[rx] = old_rank_rx
```

Use Case

Offline dynamic graph queries where edges are added and removed.
With rollback DSU + divide and conquer → **$O(E \log^2 V)$** per query.



Critical constraint: Do NOT use path compression with rollback DSU. Path compression permanently alters the tree structure, making it impossible to restore previous states. Union by rank alone gives $O(\log n)$ — acceptable for this use case.

Section 8: Borůvka's Algorithm – The Parallel MST

Borůvka's algorithm predates both Prim's and Kruskal's (1926) and is naturally parallel. Each component simultaneously finds its minimum outgoing edge and merges.

BORUVKA(G, w):

Components = {each vertex its own component}

WHILE more than 1 component:

cheapest = {}

FOR each edge (u, v, w):

Cu = **component_of**(u); Cv = **component_of**(v)

IF Cu != Cv:

IF w < cheapest.get(Cu, ∞): cheapest[Cu] = (u,v,w)

IF w < cheapest.get(Cv, ∞): cheapest[Cv] = (u,v,w)

FOR C in components:

IF cheapest[C] exists:

add cheapest[C] to MST; UNION components

RETURN MST

// Each phase: reduces component count by $\geq$ half

// Number of phases: $O(\log V)$

// Each phase: $O(E)$ edge scan

// Total: $O(E \log V)$

Key Advantage: Embarassingly Parallel



Multi-Core Machines

Each thread handles one component's cheapest edge search in parallel.



MapReduce / Distributed

Each mapper handles a partition of edges. Basis for distributed MST at scale.



Randomised MST

Foundation of the Karger-Klein-Tarjan randomised $O(E)$ MST algorithm.

Section 9: Real-World Use Cases

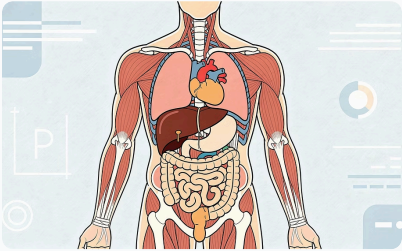
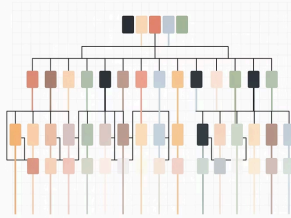


Image Segmentation via Graph Cuts

Context: Computer vision — object detection, medical image analysis.

Model: Pixels = vertices; adjacent pixels = edges (weight = colour difference). Kruskal's MST, then remove the $k-1$ heaviest MST edges to get k segments.

Why Kruskal's: Edge list naturally stores pixel adjacencies; sorting by colour difference is the bottleneck. DSU efficiently tracks merged regions.



Single-Linkage Hierarchical Clustering

Context: Bioinformatics (gene clustering), market segmentation.

Key insight: Kruskal's MST edge insertion order IS the single-linkage dendrogram. The k -th edge added is the merge event at height $w(k)$ in the cluster hierarchy. Cutting the dendrogram at height h is equivalent to removing MST edges with weight $> h$.



Road Network Planning in Sparse Terrain

Context: Civil engineering — connecting remote villages.

Why Kruskal's: Road network graphs are very sparse ($E \ll V^2$). Sorting edges by cost is the natural first step in any infrastructure cost analysis. DSU efficiently tracks connected components as roads are planned.

Section 10: Hands-On Labs

Lab 1: Full Kruskal's Implementation with DSU

Graph: 7 vertices, edges: (0,1,7), (0,3,5), (1,2,8), (1,3,9), (1,4,7), (2,4,5), (3,4,15), (3,5,6), (4,5,8), (4,6,9), (5,6,11)

1. Implement complete DSU with path compression + union by rank
2. Implement Kruskal's using the DSU
3. Trace the algorithm: for each edge in sorted order, record edge, FIND(u), FIND(v), action (ADD/SKIP), DSU state after
4. Verify final MST has $V-1 = 6$ edges
5. Count: how many FIND calls were made? How many UNION calls?

✓ **Expected MST total weight: 39.** Verify edges include $\{(0,3,5), (2,4,5), (3,5,6), (0,1,7), (1,4,7), (4,6,9)\}$ or similar.

Lab 2: Compare Kruskal's and Prim's on the Same Graph

1. Use the graph from Lab 1
2. Run both Kruskal's and Prim's (starting from vertex 0)
3. Do they produce the same MST? Same weight? (Should: same weight, possibly different edge sets if ties)
4. Measure operations: Kruskal's (sort + FIND/UNION calls) vs Prim's (heap push/pop + key comparisons)
5. Add duplicate weight edges (make two edges weight 7). Do multiple valid MSTs now exist? Can you enumerate them?
6. Increase graph density by adding 10 more random edges. At what density (E/V ratio) does Prim's $O(V^2)$ variant become faster than Kruskal's $O(E \log E)$?

Section 11: Side-by-Side Comparison

Prim's vs Kruskal's — Complete Analysis

Dimension	Prim's Algorithm	Kruskal's Algorithm
Strategy	Grow one tree from a vertex	Merge components globally
Data Structure	Min-Heap (or array)	Union-Find (DSU) + sorted edges
Processing Order	Cheapest edge out of current tree	Globally cheapest edge
Sparse ($E \approx V$)	$O(V \log V)$	$O(V \log V)$ — similar
Dense ($E \approx V^2$)	$O(V^2)$ with array scan ← faster	$O(V^2 \log V)$
Pre-sorted edges	No improvement	$O(E \alpha(V)) \approx O(E)$ ← faster
Disconnected graph	Requires outer loop per component	Naturally handles forests
Parallelisable?	Hard (sequential by nature)	✅ Via Borůvka's parallel variant
Real-world preference	Dense graphs, geometric networks	Sparse graphs, pre-sorted data
Correctness basis	Cut Property	Cut + Cycle Property

📌 When they disagree on edge selection (ties): If two edges have the same weight, Prim's and Kruskal's may choose different edges, producing different MSTs. Both MSTs always have the same **total weight**. Example: Triangle A-B-C all weight 5 → any 2 edges form a valid MST of weight 10.

Section 16: Common Mistakes – Kruskal's

INTERMEDIATE

Not Using Union by Rank in DSU

Impact: $O(n)$ per union in worst case $\rightarrow$ Kruskal's becomes $O(EV)$.

Fix: Always union by rank (or size).

INTERMEDIATE

Path Compression Without Rank

Impact: Still $O(\log n)$ without rank; need both for $O(\alpha(n))$.

Fix: Use BOTH path compression AND union by rank.

INTERMEDIATE

Checking `parent[u] == parent[v]` Instead of `FIND(u) == FIND(v)`

Impact: Misses components where representative differs from parent.

Fix: Always use `FIND()` which follows paths to root.

ADVANCED

Assuming Kruskal's Gives Wrong Result on Equal-Weight Edges

Impact: Confusion between "A MST" and "THE MST".

Fix: Any valid ordering of equal-weight edges gives A valid MST; all have the same total weight.

ADVANCED

Applying Kruskal's to Directed Graphs

Impact: MST is defined for undirected only; algorithm breaks.

Fix: For directed MST, use Edmonds' (Chu-Liu/Edmonds') algorithm.

ADVANCED

Not Breaking Early When `|A| == V-1`

Impact: Unnecessary processing of remaining edges after MST is complete.

Fix: Add `IF |A| == V-1: BREAK` inside the edge loop.

Section 18: Learning Summary

Key Takeaways — Kruskal's

- Sort all edges by weight; process cheapest first
- Add edge if endpoints in different components; skip otherwise (no cycle created)
- DSU with path compression + union by rank gives near- $O(1)$ per operation
- Total complexity: $O(E \log E) = O(E \log V)$ — dominated by sort
- Naturally handles disconnected graphs → minimum spanning forest
- Pre-sorted edges: $O(E \alpha(V)) \approx O(E)$ — near-linear!

Key Takeaways — Overall MST

- Both Prim's and Kruskal's are greedy algorithms provably correct by the Cut Property
- Prim's: vertex-centric, grows one tree, better for dense graphs with $O(V^2)$ array-scan
- Kruskal's: edge-centric, merges components, better for sparse graphs and pre-sorted data
- Borůvka's: component-centric, naturally parallel, basis for distributed/randomised MST
- MST total weight is unique; specific MST edge set may not be if weights repeat

Common Interview Questions

1. Explain Kruskal's algorithm. What data structure makes it efficient?
2. Why does Kruskal's skip edges that would form a cycle? Which MST theorem justifies this?
3. Explain Union-Find with path compression and union by rank. What is $\alpha(n)$?
4. What is the time complexity of Kruskal's? How does it change if edges are pre-sorted?
5. Can Kruskal's handle a disconnected graph? What does it produce?
6. Prim's vs Kruskal's — when would you prefer each?
7. How does Kruskal's relate to single-linkage hierarchical clustering?
8. What is Borůvka's algorithm? Why is it useful for parallel computing?
9. If two edges have equal weight, can Kruskal's produce different MSTs? Do they have the same total weight?
10. How would you modify Kruskal's to find the Maximum Spanning Tree?

📌 **Kruskal's Quick Reference:** Strategy = Sort edges, add if no cycle, use DSU to detect. Complexity = $O(E \log E)$. Pre-sorted = $O(E \alpha(V)) \approx O(E)$. Dense graph → Prim's $O(V^2)$ wins. Parallel → Borůvka's wins.